Applied Industrial Data Science

Week 4, Lecture 4: Reproducible Project Architecture and Repository Hygiene

Instructor: Dr. Muhammad Shoaib
Institution: GC University Faisalabad
Professional Role: Data Scientist, Utopi Ltd., United Kingdom

1

WEEK 4

09:00. You are not in. 09:30. The report is due.

A colleague has your repository and one hour. Do they get your number?

2

Three verdicts, and what they judge

01 Reproduces
Another person, another machine, the same answer. Nothing to ask.

02 Reproduces with help
It runs, but only after someone tells them something.

03 Does not reproduce
The answer cannot be obtained again by anyone, including you.

A verdict is about a person on a machine, not about tidiness.

Three repositories follow. Every one belongs to somebody competent, and every one works on its author's laptop.

3

A repository that works perfectly

In the repository	Size	Committed
analysis_final_v3.ipynb	2.1 MB	yes
analysis_final_v2.ipynb	1.9 MB	yes
plant_shift_log.csv	41 MB	yes
results_MARCH.xlsx	890 KB	yes
scratch_notes.txt	2 KB	yes

There is no README. It produces the right number on the author's laptop. Reproduces, reproduces with help, or does not reproduce?

4

Five things that exist only on one laptop

What is missing	What your colleague sees	What it costs
The run order	Cells numbered 1, 4, 2, 9	It has never been run top to bottom
The environment	Nothing records it	They will not get what the author had
The entry point	Two notebooks, no README	A wrong guess still produces a number
The provenance (Origin)	A 41 MB csv, no source	No source, no licence, no fingerprint
The link to the output	results_MARCH.xlsx	No way to tell which code made it

Verdict: does not reproduce. Nothing here is careless. It is simply all in one person's head.

05

5

The same project, tidied up

What it is	Path	Committed
Week 3's checks	src/validate_data.py	yes
The evidence	data/raw/plant_shift_log.csv	yes
The findings	docs/data_quality_report.md	yes
The environment	requirements.txt	yes
How to run it	README.md	yes
Ignore rules	.gitignore	no such file

Folders, a README, an environment file. One line inside src/ decides whether any of it works, and you cannot see it from here.

06

6

One line that cannot run anywhere else

In the code	Author's machine	Anyone else's
read_csv("C:/Users/Shoaib/plant.csv")	Works	FileNotFoundError, immediately
open("C:/Users/Shoaib/out.json")	Works	Fails, or writes somewhere nobody looks
read_csv("data/raw/plant_shift_log.csv")	Works	Works, from a fresh clone, anywhere

A path is configuration, not code.

Anything that differs between two machines belongs outside the code that uses it: a config file, or an argument you pass in.

07

7

It even has a requirements file

requirements.txt	Installed in March	Installed in September
pandas	2.2.3	2.3.1
numpy	2.1.3	2.3.2
pytest	8.3.4	8.4.1
ipykernel	6.29.5	7.0.1
matplotlib	3.9.2	3.10.5

Same file, same command, six months apart. Reproduces, reproduces with help, or does not reproduce?

08

8

A name is not a version

What you write	What you get	What it promises
pandas	Newest today	Nothing at all
pandas==2.2.3	2.2.3, always	The same answer, or a clear failure

An environment file is a claim about software, not a shopping list.

Pin what you install, and record how you installed it. Both belong in the repository, not in your memory.

09

9

What each folder promises

Path	The promise	What breaks without it
data/raw/	Never edited, never overwritten	Nothing can settle a dispute later
data/processed/	Derived, and rebuildable from raw	Nobody can tell input from output
src/	Code that can be imported and tested	The logic lives in cells nobody can call
tests/	The checks run without you	Breakage is found by a person
docs/	Decisions, in writing	The reasoning leaves with the author
artifacts/	Generated, never hand-edited	Evidence and output blur together

The structure is not tidiness. Each folder is a promise, and the promise is what makes the result checkable.

10

10

The same code, somewhere it can be tested

In a notebook	In src/	Why it matters
A cell you scroll to find	A named function	Something other than you can call it
State left by earlier cells	Arguments in	The result stops depending on run order
Checked by reading the output	Checked by a test	It is still checked in six months

The notebook becomes where you explore, not where the work lives.

Move it once it is right twice. A notebook is the correct tool, right up until somebody else needs the answer.

11

11

What changes between machines

The thing	Must not live	Where it belongs
Paths and filenames	In the code	A config file, or an argument you pass in
Credentials and tokens	In the repo	Outside the repository, in the environment

If it differs between two machines, it is configuration.

A repository that must be edited before it runs has not been handed over. It has been lent.

12

12

What must never reach the remote

Pattern	What it keeps out	Why
.venv/	The environment itself	Large, and wrong on other machines
__pycache__/	Compiled bytecode	Regenerated every run
.env, secrets.	Credentials	A pushed token is a published token
data/raw/*	Large or restricted evidence	Size and licence; commit a fingerprint
artifacts/, *.xlsx	Generated results	The code makes them; two copies drift
.ipynb_checkpoints/	Notebook autosaves	Noise in every diff, for ever

A .gitignore is not housekeeping. It is the list of things you have decided not to publish.

13

13

The commit you cannot take back

What happened	When	What is true now
An API key committed in config.py	Commit 4 of 60	Readable by anyone who can clone
Spotted, file deleted, committed again	Commit 5 of 60	Still readable, in full, in commit 4
Repository switched to private	Three weeks later	The key was public for three weeks

Deleting a file removes it from the present, not from the history.

Git is built to remember. That is the entire point of it, and it is why the fix is never a delete.

14

14

The only fix that works

The instinct	What it achieves	What to do instead
Delete the file and commit	Hides one version	Rotate the credential. Assume it is known.
Rewrite the history	Copies survive	Rotate first, rewrite after, tell everyone.

Treat a pushed secret as a published secret.
The clock started when you pushed.

Never put a token, password or recovery code in a notebook, a commit message, or any file in the repository. There is no undo.

15

15

A commit message answers one question

The question it must answer	Weak	Strong
Why did this change?	update	Reject negative energy, do not drop the row
Can I find it later?	fix	Add coverage check for missing hours
Is this one change?	final v3, misc	One change, one commit

Why, not what. The diff already says what.

You are writing to whoever has to understand this in March, and that person is usually you.

16

16

The markers are not damage

What you see	What it means	What it is not
<<<<<<< HEAD	Your version starts	A corrupted file
=======	Theirs starts below	A line to keep, it must go too
>>>>>>> feature-branch	Theirs ends here	A reason to delete their branch

Git stopped because two people changed the same lines. It is asking, not failing.

Read both sides, choose or write a third version, delete all three markers, then run the tests before you commit.

17

17

The reproducibility checklist

Check	The question it asks	The evidence in the repository
Environment	Can the software be reinstalled exactly?	requirements.txt, with pinned versions
Entry point	Is it obvious what to run?	README, with one command
Paths	Does it run from a fresh clone?	No absolute path anywhere in src/
Data	Is the source and licence recorded?	docs/, with the fingerprint from Week 3
Secrets	Is anything private committed?	.gitignore, and a checked history
Tests	Does anything check it without you?	tests/ that fail when they should

This is what the lab produces, and it is exactly how the capstone repository will be marked.

18

18

EXIT REFLECTION

Could a stranger get your number/result tomorrow?

Not whether it is tidy. Whether the answer survives without you in the room.

19

19

In the lab, you hand your repository over

IN THE LAB

Restructure your Week 3 work into the skeleton
Pin the environment and write the README
Write the three missing checks until the tests pass
Clone it cold, and complete the checklist
Then run a partner's repository, cold

WEEK 5

Baseline modelling
A number to compare against, and a residual you have to explain

Bring a repository a stranger can run, and one thing you still had to tell them.

The second one is what the checklist is for.

20

20